

UNIVERSIDAD CONTINENTAL

FACULTAD DE INGENIERIA
ESCUELA ACADEMICO PROFESIONAL DE INGENIERIA DE SISTEMAS E INFORMATICA

INFORME TECNICO DE EXAMEN PARCIAL ESTRUCTURA DE DATOS

CASO PRACTICO: GESTION DE UNA FLOTA DE VEHICULOS ELECTRICOS
Tipo D - Evaluacion Individual

ASIGNATURA:	Estructura de Datos
DOCENTE:	Dr. Ing. Julio Arboleda H.
SECCION:	24UC00428
FECHA DE ENTREGA:	05 de Octubre de 2026
LENGUAJE:	Python 3 (Estandar Estructurado)
ESCALA DE CALIFICACION:	0 a 20 puntos (12 ptsCodigo + 8 pts Informe)

1. Introduccion

1.1. Descripcion del Sistema

En la gestion moderna de operaciones de transporte, las empresas de alquiler de vehiculos electricos enfrentan retos computacionales y logísticos vinculados a la autonomia de bateria. A diferencia de las unidades de combustion interna, el rendimiento de un vehiculo electrico depende de la capacidad neta de almacenamiento (kWh) y de la tasa de consumo promedio (kWh/100km). El presente sistema ha sido desarrollado en Python siguiendo un diseno modular y estructurado, permitiendo administrar el inventario, monitorear el kilometraje y generar reportes de desempeno.

1.2. Objetivo del Programa

El objetivo primordial es automatizar el ciclo operativo de la flota: registro controlado sin duplicidad de ID, actualizacion de servicios completados, conmutacion de estados de disponibilidad (Disponible vs. En mantenimiento), calculo matematico en tiempo real de la autonomia estimada y emision de reportes analiticos para priorizar recargas y mantenimientos.

1.3. Justificacion del Uso de Estructuras (struct) y Arreglos Unidimensionales en Python

En Python, los conceptos clasicos de registros y arreglos estaticos se implementan mediante abstracciones nativas coherentes:

- **Registros (struct) mediante Clases de Atributos Puros:** Un vehiculo electrico concentra datos heterogeneos (entero para ID, cadena para modelo, flotantes para bateria y consumo, enteros para viajes y estado). Se implemento 'class Vehiculo' exclusivamente como un registro estructurado sin metodos complejos, garantizando encapsulamiento, legibilidad y cohesión conceptual.
- **Arreglos Unidimensionales de Capacidad Estatica:** Se inicializa un arreglo de tamano prefijado 'flota = [None] * MAX_VEHICULOS' gobernado por una variable 'total_vehiculos'. Esta tecnica replica con exactitud el comportamiento de los arreglos estaticos de memoria contigua, permitiendo acceso O(1) y sirviendo de base para la aplicacion manual del ordenamiento por burbuja y busqueda secuencial.

ENFOQUE PEDAGOGICO EN PYTHON: La implementacion de 'class Vehiculo' como registro de campos puros y la reserva previa de memoria con '[None] * MAX_VEHICULOS' permite estudiar y demostrar la mecanica de estructuras de datos estaticas sin recurrir a metodos opacos de alto nivel.

2. Logica del Ingreso de Datos

2.1. Flujo de Captura de Datos

El flujo de captura asegura que ninguna unidad con datos incongruentes se registre en el sistema:

- **1. Verificacion de capacidad:** Se valida preliminarmente que 'total < MAX_VEHICULOS'.
- **2. Captura y validacion de ID unico:** Se lee el ID y se ejecuta una busqueda secuencial para garantizar que no exista colision.
- **3. Captura del modelo:** Se captura la cadena permitiendo espacios y validando que no quede en blanco.
- **4. Captura de bateria y consumo:** Se solicitan valores numericos mayores que cero (> 0).

- **5. Captura de viajes y estado:** Se valida que los viajes sean no negativos (≥ 0) y que el estado sea estrictamente 1 o 2.
- **6. Insercion:** Se instancia 'Vehiculo(...)' en 'flota[total]' y se incrementa el contador.

2.2. Validaciones Implementadas

- **ID Unico:** La funcion 'buscar_vehiculo_por_id' debe retornar -1. Si retorna una posicion valida, se bloquea la insercion.
- **Capacidad y Consumo (> 0):** Representan cantidades fisicas que deben ser estrictamente positivas.
- **Viajes Realizados (≥ 0):** El historial de viajes no puede ser negativo.
- **Estado (1 o 2):** Solo se aceptan las opciones 1 (Disponible) o 2 (En mantenimiento).

2.3. Uso de Estructuras de Control (WHILE, IF, FOR)

Se implementan bucles interactivos 'while True' complementados con sentencias condicionales 'if-else'. Esto retiene al usuario en el campo erroneo hasta que proporcione un valor admisible, impidiendo el avance con datos corruptos.

2.4. Manejo de Errores (Ingreso de Letras en Campos Numericos)

En Python, convertir una cadena con letras a entero o flotante genera una excepcion 'ValueError'. Para evitar que el programa aborte, se implementaron funciones de lectura con bloques 'try-except':

```
def leer_entero(mensaje: str) -> int:
    while True:
        entrada = input(mensaje).strip()
        try:
            return int(entrada)
        except ValueError:
            print("    [Error] Entrada invalida. Debe ingresar un numero entero.")

def leer_float_positivo(mensaje: str) -> float:
    while True:
        entrada = input(mensaje).strip()
        try:
            valor = float(entrada)
            if valor > 0.0:
                return valor
            print("    [Error] El valor debe ser estrictamente mayor que cero (> 0).")
        except ValueError:
            print("    [Error] Entrada invalida. Ingrese un valor numerico valido.")
```

Fragmento: Control de excepciones ante entrada de texto en campos numericos

3. Logica de Almacenamiento

3.1. Uso del Arreglo Unidimensional para Almacenar Vehiculos

La flota se almacena en la estructura 'flota = [None] * MAX_VEHICULOS'. Cada celda almacena la referencia a una instancia del registro 'Vehiculo'. Al indexarse mediante posiciones contiguas [0, MAX_VEHICULOS - 1], se accede a cualquier unidad en tiempo $O(1)$.

3.2. Control del Numero de Elementos (Variable Contador)

La variable 'total_vehiculos' gestiona el tamaño lógico de la colección:

- **Índice de asignación:** Determina la posición de escritura libre: 'flota[total] = nuevo'.
- **Frontera de iteración:** Define el rango útil 'range(total)' para búsquedas, listados, cálculos y ordenamiento.
- **Barrera de desbordamiento:** Permite evaluar 'total >= MAX_VEHICULOS' antes de permitir nuevos registros.

3.3. Limitaciones del Enfoque Estático

- **Capacidad Máxima Fija:** Al alcanzar las 100 unidades, el sistema debe rechazar nuevos ingresos a menos que se modifique el código.
- **Memoria Reservada Ociosa:** Si la flota solo tiene 5 vehículos, los 95 elementos restantes permanecen reservados como 'None'.
- **Falta de Elasticidad:** No permite crecimiento dinámico automático bajo demanda en tiempo de ejecución.

4. Lógica de Actualización

4.1. Procedimiento para Actualizar el Numero de Viajes

La función 'actualizar_viajes' localiza el vehículo existente y modifica su atributo 'viajes_realizados' in-place sin recrear la entidad.

4.2. Validaciones Previas

- **Flota con registros:** Comprueba que existan unidades cargadas ('total > 0').
- **Existencia de unidad:** Se busca el ID ingresado. Si retorna -1, se emite una notificación y se aborta.
- **Valor no negativo:** El nuevo valor debe ser mayor o igual a cero (≥ 0).

4.3. Uso de Búsqueda Secuencial para Localización

Se aprovecha el índice devuelto por la búsqueda lineal: 'flota[indice].viajes_realizados = nuevos_viajes', logrando actualización inmediata $O(1)$.

5. Lógica de Ordenación

5.1. Algoritmo Usado: Metodo de la Burbuja (Bubble Sort)

Se implementó el algoritmo clásico de ordenamiento por intercambio (Burbuja) de forma manual y explícita, sin utilizar métodos preconstruidos de Python como '.sort()' o 'sorted()'. Efectúa pasadas sucesivas comparando elementos adyacentes y trasladando los valores menores hacia el final.

5.2. Proceso de Ordenación por Autonomía (Cálculo Derivado)

El ordenamiento es de mayor a menor (descendente). Dado que la autonomía no es un dato estático sino derivado, el algoritmo evalúa la función 'calcular_autonomia()' sobre los elementos contiguos en cada iteración interna:

```
def ordenar_vehiculos_por_autonomia(flota: list, total: int):
```

```

for i in range(total - 1):
    for j in range(total - 1 - i):
        auto_actual = calcular_autonomia(flota[j])
        auto_siguiente = calcular_autonomia(flota[j + 1])

        # Criterio descendente: desplaza los valores mayores hacia el inicio
        if auto_actual < auto_siguiente:
            flota[j], flota[j + 1] = flota[j + 1], flota[j]

```

Fragmento: Metodo de la Burbuja manual descendente sobre calculo derivado en Python

5.3. Justificacion Tecnica de Ordenar sobre el Calculo vs. Datos Crudos

- **Consistencia y Fuente Unica de Verdad:** Evita almacenar un campo redundante que podria desincronizarse si se alteran la bateria o el consumo.
- **Evaluacion Multivariable:** La autonomia combina dos magnitudes independientes. Una bateria grande con alto consumo puede rendir menos que una bateria mediana eficiente. Ordenar sobre el calculo es la unica manera de clasificar el rendimiento real.

6. Logica de Búsqueda

6.1. Búsqueda Secuencial por ID

La funcion 'buscar_vehiculo_por_id' recorre iterativamente el arreglo desde el indice 0 hasta 'total - 1', comparando el campo 'id' de cada vehiculo con el valor buscado.

6.2. Casos Posibles

- **Encontrado:** Retorna el indice [0, total-1] para acceso directo a sus campos.
- **No Encontrado:** Retorna -1 tras agotar el recorrido util, activando advertencias en la interfaz.
- **ID Duplicado:** Se intercepta en el alta de vehiculos; si la busqueda retorna distinto de -1, se prohíbe el registro.

6.3. Mejora Potencial: Búsqueda Binaria

Si los vehiculos se mantuviesen ordenados por ID, se podria implementar la Búsqueda Binaria. Esto reduciria la complejidad de $O(n)$ a $O(\log n)$. Para una flota de 100 unidades, el peor caso pasaria de 100 comparaciones a un maximo de 7.

7. Logica de Calculos

7.1. Formula Matematica de Autonomia Estimada

La autonomia expresa los kilometros que el vehiculo puede recorrer con su carga completa:

$$\text{Autonomia (km)} = (\text{Capacidad de Bateria (kWh)} / \text{Consumo Promedio (kWh/100km)}) * 100$$

7.2. Uso de Acumuladores (Promedio General de la Flota)

En la opcion 6 ('consultar_autonomia'), el sistema suma la autonomia de todos los vehiculos activos mediante un acumulador y calcula la media aritmetica dividiendo entre 'total_vehiculos', ofreciendo un indicador global de la flota.

7.3. Validaciones (Prevencion de Division por Cero)

Para blindar la division contra errores aritmeticos en caso de consumos anomalos, se implemento una clausula de guarda:

```
def calcular_autonomia(v: Vehiculo) -> float:
    if v.consumo_promedio <= 0.0:
        return 0.0 # Previene indeterminaciones por division entre cero
    return (v.capacidad_bateria / v.consumo_promedio) * 100.0
```

Fragmento: Clausula de guarda para evitar division por cero

8. Logica de Reportes

8.1. Condiciones y Umbrales Dinamicos

- **Reporte de Autonomia Baja:** Condicion: 'calcular_autonomia(v) < umbral_km'. Identifica unidades que precisan recarga inmediata.
- **Reporte de Alta Demanda:** Condicion: 'v.viajes_realizados > umbral_viajes'. Identifica unidades con alto desgaste para programar mantenimiento.

8.2. Filtrado y Presentacion Tabular

Ambos reportes aplican un patron de filtrado con conteo de coincidencias ('encontrados'). Los resultados se formatean en tabla mediante f-strings alineadas con anchos fijos, emitiendo mensajes claros si ningun elemento coincide.

9. Logica del Menu

9.1. Implementacion del Menu

El menu despliega las 9 opciones solicitadas. Cada operacion esta delegada en su respectiva funcion modular.

9.2. Uso de Bucles ('while')

Se utiliza un bucle 'while opcion != 9'. Esta estructura asegura que la interfaz se muestre de manera continua tras completar cada accion hasta que el operador digite la opcion de salida.

9.3. Manejo de Opciones Invalidas

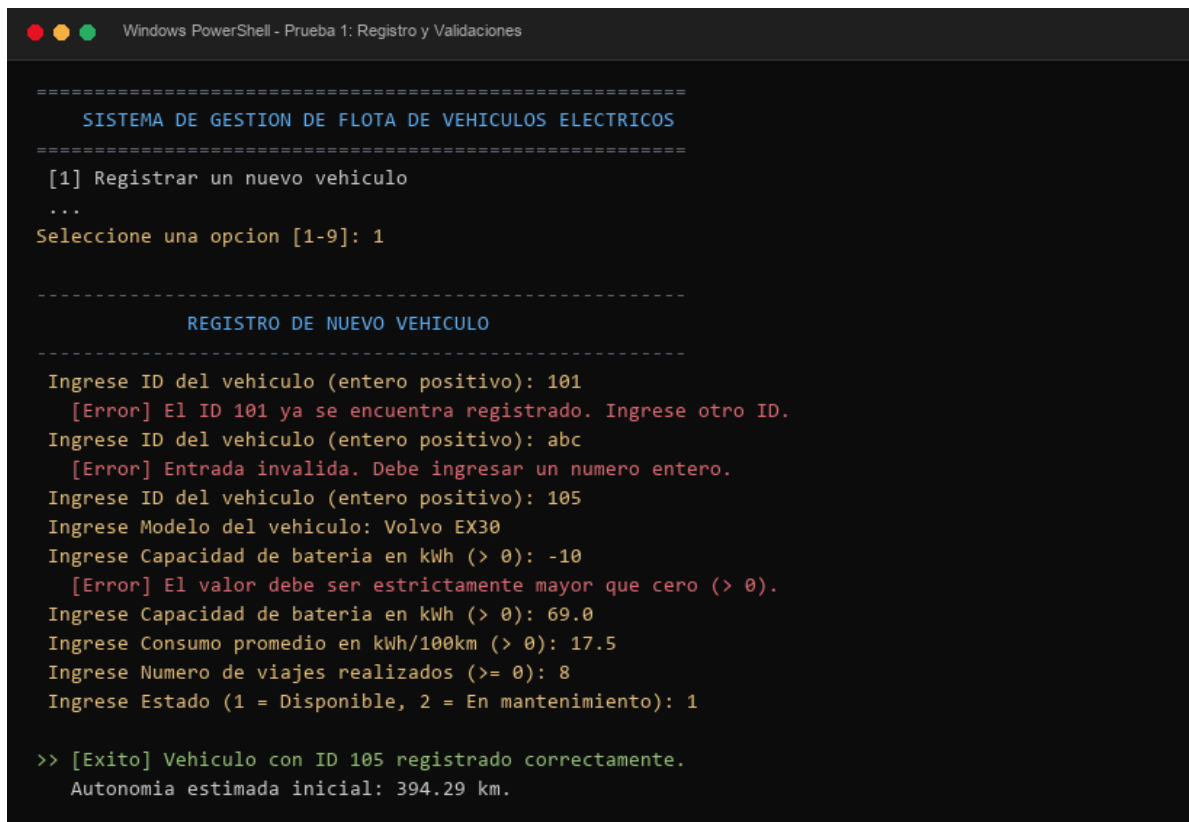
La captura de opcion utiliza 'leer_entero_mayor_igual', lo que neutraliza caracteres alfabeticos. Adicionalmente, una clausula 'else' final notifica cuando se digita un numero fuera del rango [1, 9].

10. Pruebas del Sistema (Evidencias y Resultados)

A continuacion se presentan los casos de prueba ejecutados sobre el sistema en Python, acreditando el cumplimiento de las especificaciones tecnicas.

10.1. Caso de Prueba 1: Registro con Validacion de Errores

- **Objetivo:** Comprobar rechazo de duplicados, control de caracteres alfabeticos y cotas numericas.
- **Valores probados:** ID duplicado = 101; ID letra = 'abc'; ID valido = 105; Capacidad -10 kWh; Capacidad valida = 69.0 kWh; Consumo = 17.5; Viajes = 8; Estado = 1.
- **Resultado:** Registro exitoso del Volvo EX30 con autonomia inicial calculada de 394.29 km.



```

Windows PowerShell - Prueba 1: Registro y Validaciones

=====
      SISTEMA DE GESTION DE FLOTA DE VEHICULOS ELECTRICOS
=====

[1] Registrar un nuevo vehiculo
...
Seleccione una opcion [1-9]: 1

-----
      REGISTRO DE NUEVO VEHICULO
-----

Ingrese ID del vehiculo (entero positivo): 101
[Error] El ID 101 ya se encuentra registrado. Ingrese otro ID.
Ingrese ID del vehiculo (entero positivo): abc
[Error] Entrada invalida. Debe ingresar un numero entero.
Ingrese ID del vehiculo (entero positivo): 105
Ingrese Modelo del vehiculo: Volvo EX30
Ingrese Capacidad de bateria en kWh (> 0): -10
[Error] El valor debe ser estrictamente mayor que cero (> 0).
Ingrese Capacidad de bateria en kWh (> 0): 69.0
Ingrese Consumo promedio en kWh/100km (> 0): 17.5
Ingrese Numero de viajes realizados (>= 0): 8
Ingrese Estado (1 = Disponible, 2 = En mantenimiento): 1

>> [Exito] Vehiculo con ID 105 registrado correctamente.
Autonomia estimada inicial: 394.29 km.
  
```

Figura 1: Evidencia de validaciones en registro y control de excepciones.

10.2. Caso de Prueba 2: Busqueda Secuencial por ID

- **Objetivo:** Comprobar respuesta ante identificadores existentes e inexistentes.
- **Valores probados:** ID existente = 103 (BYD Han EV); ID inexistente = 999.
- **Resultado:** Recuperacion precisa de datos y aviso claro ante clave no encontrada.

```

Windows PowerShell - Prueba 2: Busqueda Secuencial por ID

-----
BUSQUEDA DE VEHICULO POR ID
-----

Ingrese el ID del vehiculo a buscar: 103

>> Vehiculo encontrado exitosamente (Posicion en arreglo: 2):
- ID : 103
- Modelo : BYD Han EV
- Capacidad bateria : 85.40 kWh
- Consumo promedio : 18.20 kWh/100km
- Viajes realizados : 40
- Estado actual : En mantenimiento
- Autonomia calculada: 469.23 km

Ingrese el ID del vehiculo a buscar: 999

[!] No se encontro ningun vehiculo con el ID 999.

```

Figura 2: Evidencia de busqueda secuencial exitosa y manejo de registro inexistente.

10.3. Caso de Prueba 3: Ordenacion por Autonomia Estimada (Burbuja)

- **Objetivo:** Comprobar que el algoritmo Burbuja clasifica la flota de mayor a menor autonomia.
- **Resultado:** Ordenamiento descendente: BYD Han EV (469.23 km), Ioniq 5 (427.06 km), Model 3 (400.00 km), Volvo EX30 (394.29 km), Leaf (242.42 km).

```

Windows PowerShell - Prueba 3: Ordenamiento Burbuja y Listado

-----
ORDENAR VEHICULOS POR AUTONOMIA (MAYOR A MENOR)
-----

>> [Exito] Flota ordenada exitosamente por autonomia estimada (descendente).

-----
LISTADO GENERAL DE LA FLOTA
-----

+-----+-----+-----+-----+-----+-----+-----+
| ID | Modelo | Bateria | Consumo | Viajes | Estado | Autonomia Est. |
+-----+-----+-----+-----+-----+-----+-----+
| 103 | BYD Han EV | 85.4 kWh | 18.2 km | 40 | Mantenimiento | 469.23 km |
| 104 | Hyundai Ioniq 5 | 72.6 kWh | 17.0 km | 30 | Disponible | 427.06 km |
| 101 | Tesla Model 3 | 60.0 kWh | 15.0 km | 25 | Disponible | 400.00 km |
| 105 | Volvo EX30 | 69.0 kWh | 17.5 km | 8 | Disponible | 394.29 km |
| 102 | Nissan Leaf | 40.0 kWh | 16.5 km | 12 | Disponible | 242.42 km |
+-----+-----+-----+-----+-----+-----+-----+

Total de unidades registradas: 5 / 100

```

Figura 3: Evidencia del ordenamiento descendente por autonomia calculada.

10.4. Caso de Prueba 4: Generacion de Reportes por Umbral

- **Objetivo:** Evaluar filtros de baja autonomia (< umbral) y alta demanda (> umbral).

- **Umbrales probados:** Autonomia < 350.0 km (filtro: Nissan Leaf con 242.42 km); Viajes > 25 (filtro: BYD Han con 40 y Hyundai Ioniq 5 con 30).

```
Windows PowerShell - Prueba 4: Generacion de Reportes por Umbral

-----
MODULO DE REPORTES DE RENDIMIENTO
-----

[1] Reporte de vehiculos con AUTONOMIA BAJA (Autonomia < umbral)
[2] Reporte de vehiculos con ALTA DEMANDA (Viajes > umbral)
Seleccione tipo de reporte [1 o 2]: 1
Ingrese umbral de autonomia minima en km: 350

>> VEHICULOS CON AUTONOMIA MENOR A 350.00 KM:
+-----+-----+-----+-----+-----+-----+-----+
| ID | Modelo           | Bateria   | Consumo   | Viajes | Estado       | Autonomia Est. |
+-----+-----+-----+-----+-----+-----+-----+
| 102 | Nissan Leaf      | 40.0 kWh  | 16.5 km   | 12     | Disponible   | 242.42 km      |
+-----+-----+-----+-----+-----+-----+-----+
>> Total de vehiculos identificados con autonomia baja: 1

Seleccione tipo de reporte [1 o 2]: 2
Ingrese umbral minimo de viajes completados: 25

>> VEHICULOS CON ALTA DEMANDA (VIAJES MAYOR A 25):
+-----+-----+-----+-----+-----+-----+-----+
| ID | Modelo           | Bateria   | Consumo   | Viajes | Estado       | Autonomia Est. |
+-----+-----+-----+-----+-----+-----+-----+
| 103 | BYD Han EV       | 85.4 kWh  | 18.2 km   | 40     | Mantenimiento | 469.23 km      |
| 104 | Hyundai Ioniq 5   | 72.6 kWh  | 17.0 km   | 30     | Disponible   | 427.06 km      |
+-----+-----+-----+-----+-----+-----+-----+
>> Total de vehiculos con alta demanda: 2
```

Figura 4: Evidencia de los reportes condicionales de autonomia y viajes.

10.5. Caso de Prueba 5: Actualizacion de Viajes y Cambio de Estado

- **Objetivo:** Comprobar la persistencia en memoria durante la ejecucion.
- **Acciones:** ID 102 (Nissan Leaf): viajes aumentados de 12 a 18. ID 103 (BYD Han): estado conmutado de [2] En mantenimiento a [1] Disponible.

```

Windows PowerShell - Prueba 5: Actualizacion y Cambio de Estado

-----
                ACTUALIZAR NUMERO DE VIAJES
-----

Ingrese el ID del vehiculo a actualizar: 102
Vehiculo seleccionado: Nissan Leaf
Cantidad actual de viajes: 12
Ingrese la nueva cantidad total de viajes (>= 0): 18

>> [Exito] Numero de viajes actualizado satisfactoriamente a 18.

-----
                CAMBIAR ESTADO DE OPERACION
-----

Ingrese ID del vehiculo a modificar: 103
Unidad seleccionada: BYD Han EV (ID: 103)
Estado actual: [2] En mantenimiento
Ingrese nuevo estado (1 = Disponible, 2 = En mantenimiento): 1

>> [Exito] Estado actualizado a: Disponible.

```

Figura 5: Evidencia de actualizacion de viajes y cambio de estado.

11. Oportunidades de Mejora

11.1. Reflexion sobre las Limitaciones del Sistema Estatico

El programa cumple los requerimientos curriculares de Estructura de Datos. No obstante, sufre de limitaciones: volatilidad de datos en RAM al cerrar el proceso y limite rigido de 100 vehiculos.

11.2. Propuestas Tecnicas de Evolucion

- **1. Estructuras Dinamicas:** Migrar a listas enlazadas manuales o aprovechar listas dinamicas elásticas de Python sin tope de memoria prefijado.
- **2. Persistencia en Archivos:** Integrar los modulos 'json' o 'csv' de Python para persistir la flota en disco al salir y cargarla al inicio.
- **3. Base de Datos Relacional:** Incorporar SQLite mediante el modulo nativo 'sqlite3' para soporte ACID transaccional.
- **4. Interfaz Grafica o Web:** Implementar una interfaz de usuario con Tkinter / PyQt o un dashboard con Flask / Streamlit.